5.1.1. Circular Linked list Operations

Code :

```
#include <stdio.h>

#include <stdlib.h>


struct node {

   int data;

   struct node *next;

};


typedef struct node *NODE;


// Create a new node with given data

NODE createNodeInCLL(int data) {

   NODE temp = (NODE) malloc(sizeof(struct node));

   temp->data = data;

   temp->next = NULL;

   return temp;

}


// Create CLL with n nodes

NODE createCLL( int n     ) {


// wrisdte your code here..

           if (n <= 0)

      return NULL;


   NODE first = NULL, temp = NULL, last = NULL;
```

```
    int x;

    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        temp = createNodeInCLL(x);

        if (first == NULL) {
            first = temp;
            last = temp;
            last->next = first;
        } else {
            last->next = temp;
            last = temp;
            last->next = first;
        }
    }
    return first;





}

// Traverse CLL
void traverseListInCLL( NODE first     ) {

// write your code here...
            if (first == NULL)
```

```
        return;

    NODE temp = first;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != first);
    printf("\n");





}

// Insert at given position in CLL
NODE insertAtPositionInCLL( NODE first,int pos,int x       ) {



//write your code here..
            NODE newNode = createNodeInCLL(x);

    if (first == NULL) {
        if (pos == 1) {
            newNode->next = newNode;
            return newNode;
        } else {
            printf("Position not found\n");
            return first;
```

```
    }
}

int count = 1;
NODE temp = first;

while (temp->next != first) {
    temp = temp->next;
    count++;
}

if (pos < 1 || pos > count + 1) {
    printf("Position not found\n");
    return first;
}

if (pos == 1) {
    NODE last = first;
    while (last->next != first)
        last = last->next;

    newNode->next = first;
    last->next = newNode;
    return newNode;
}

temp = first;
for (int i = 1; i < pos - 1; i++)
```

```
        temp = temp->next;

    newNode->next = temp->next;
    temp->next = newNode;

    return first;




}

// Delete node at given position in CLL
NODE deleteAtPositionInCLL( NODE first,int pos) {
            if (first == NULL) {
        printf("CLL is empty\n");
        return NULL;
    }

    int count = 1;
    NODE temp = first;

    while (temp->next != first) {
        temp = temp->next;
        count++;
    }

    if (pos < 1 || pos > count) {
        printf("Position not found\n");
```

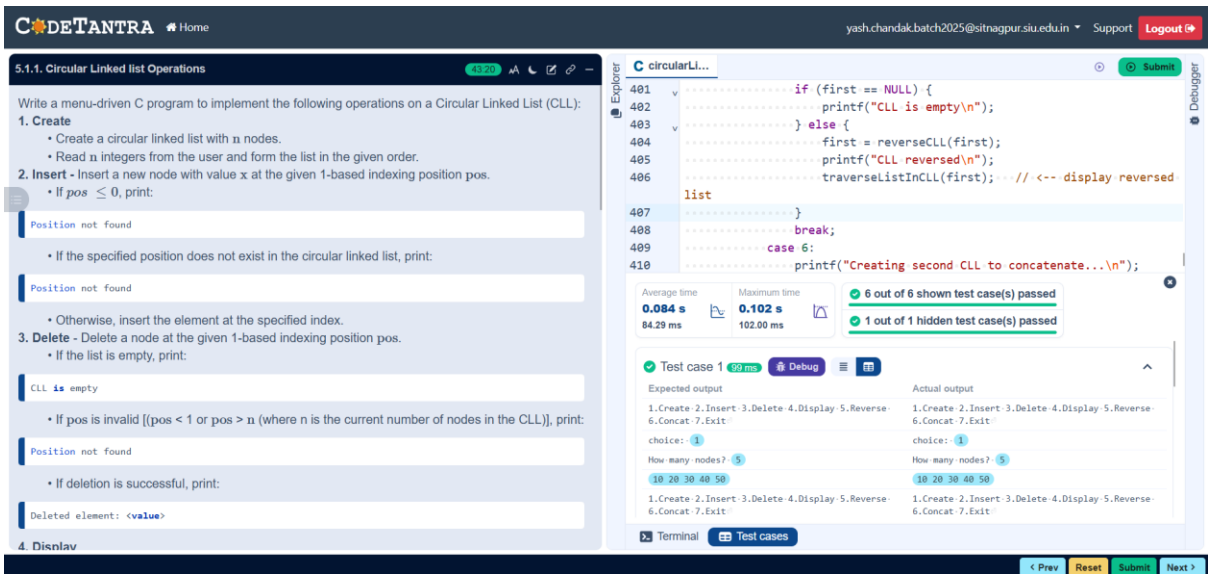
```
    return first;
}

if (pos == 1) {
    if (count == 1) {
        printf("Deleted element: %d\n", first->data);
        free(first);
        return NULL;
    }

    NODE last = first;
    while (last->next != first)
        last = last->next;

    NODE del = first;
    printf("Deleted element: %d\n", del->data);
    first = first->next;
    last->next = first;
    free(del);
    return first;
}

temp = first;
for (int i = 1; i < pos - 1; i++)
    temp = temp->next;

NODE del = temp->next;
printf("Deleted element: %d\n", del->data);
```

```
    temp->next = del->next;
    free(del);

    return first;



}

// Reverse CLL
NODE reverseCLL( NODE first      ) {




            if (first == NULL || first->next == first)
        return first;

    NODE prev = NULL, current = first, nextNode;

    do {
        nextNode = current->next;
        current->next = prev;
        prev = current;
        current = nextNode;
    } while (current != first);

    first->next = prev;
    first = prev;
```

```
    return first;



}

// Concatenate two CLLs
NODE concatCLL( NODE first,NODE second ) {

if (first == NULL)
        return second;
    if (second == NULL)
        return first;

    NODE temp1 = first;
    while (temp1->next != first)
        temp1 = temp1->next;

    NODE temp2 = second;
    while (temp2->next != second)
        temp2 = temp2->next;

    temp1->next = second;
    temp2->next = first;

    return first;
```

```
}

int main() {
  NODE first = NULL, second = NULL;
  int x, pos, op, n;

  while (1) {
    printf("1.Create 2.Insert 3.Delete 4.Display 5.Reverse 6.Concat 7.Exit\n");
    printf("choice: ");
    scanf("%d", &op);
    switch (op) {
      case 1:
        printf("How many nodes? ");
        scanf("%d", &n);
        first = createCLL(n);
        break;
      case 2:
        printf("Position: ");
        scanf("%d", &pos);
        if (pos <= 0) {
          printf("Position not found\n");
        } else {
          printf("Element: ");
          scanf("%d", &x);
          first = insertAtPositionInCLL(first, pos, x);
        }
        break;
      case 3:
```

```
    if (first == NULL) {
        printf("CLL is empty\n");
    } else {
        printf("Position: ");
        scanf("%d", &pos);
        first = deleteAtPositionInCLL(first, pos);
    }
    break;
  case 4:
    if (first == NULL) {
        printf("CLL is empty\n");
    } else {
        printf("Elements in CLL are: ");
        traverseListInCLL(first);
    }
    break;
  case 5:
    if (first == NULL) {
        printf("CLL is empty\n");
    } else {
        first = reverseCLL(first);
        printf("CLL reversed\n");
        traverseListInCLL(first);   // <-- display reversed list
    }
    break;
  case 6:
    printf("Creating second CLL to concatenate...\n");
    printf("How many nodes in second CLL? ");
```

```
            scanf("%d", &n);
            second = createCLL(n);
            first = concatCLL(first, second);
            printf("Concatenated CLL:\n");
            traverseListInCLL(first);
            break;
        case 7:
            exit(0);
    }
  }
}
```

Programme :